# 29th March Auth with Typescript, Drizzle & Postgres

— Jaani

1

## Authentication Service

⇒ Postgres is a relational database, with tables & columns.

⇒ Data will be kept in a strict format.

Ways to hosting PostgresDB Hosting (online ones)

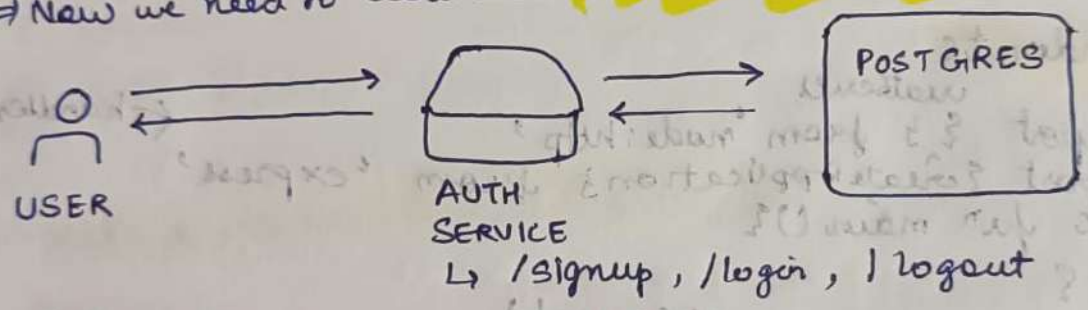
→ Supabase

→ AWS RDMS – $ 1000 per month

⇒ Neon DB

We can locally install postgres but that would bloat our system, so we prefer DOCKER

⇒ Now we need to download DOCKER DESKTOP




ORM – Object Relational Mapping, these help you interact with DB using objects instead of SQL. You can write code in any lang (JS, Python), and ORM handles everything so no need to write raw SQL.
It is also developer friendly, helps prevent SQL injection and has lot more advantages.

Drizzle ORM – This is a TS-first ORM that gives us full control like SQL (can write raw SQL) + also gives safety like TS.
It is also lightweight and fast.

pnpm is preferred over yarn; bcs it is faster, more efficient, enforces stricter dependency management.

2

Install "tsc-watch" as dev dependency.

Add few scripts:

```
"start": "node dist/index",
"build": "tsc -p .",
"dev": "tsc-watch --onSuccess \"node dist/index\"",
```

Install pnpm i @types/node, @types/express

CMD + Shift + P, + Reload VS if TS is not working

Note: uncomment the lines in tsconfig.json

Inside index.ts

```
import { createServer } from 'node:http'
import {create Application} from 'express'
async fn main (){
  try {
    const server = createServer()
    const PORT = 8050
    server.listen (PORT, () => {
        log ('Http server running on PORT ${PORT}').
    })
  }catch (error) {
        log ('Error starting http server')
        throw Error
  }
}

main ()
```

← handler

Create a app folder which stores all business logic

app ⇒ index.ts

3

```
import express from 'express'
import type {Express} from 'express'

export fun createApplication(): Express {
    const app = express()

    //Routes
    app.get('/', (req, res) => {
        return res.json({message: 'Welcome'})
    })
    return app
}
```

Using Requestly

1. Create a collection (aka folder)
2. Request :

GET : http://localhost:8080

⇒ Make a docker-compose.yml file

```
services:
  postgresdb:
    image : postgres:17
    ports : - 5432:5432
    environment : POSTGRES_USER : postgres
                  POSTGRES_PASSWORD : postgres, POSTGRES_DB : Claicode
    volumes : postgres-data : var/lib/postgresql/data.
```

hub.docker.com is a cloud platform to store, share or download Docker images. A docker image is a pre-built, read only package needed to run an application

Port Mapping : Connects a port on our local machine to a port inside Docker container

**Commands**: docker compose up

docker compose up -d (run in detached mode)

This will free up our terminal, so even if you do ctrl + C, it keeps running in background.

**Now lets make connection to DB**

Inside db create a schema.ts, also drizzle.config.js

Now follow all the steps in the postgres documentation,

so in .env,

DATABASE_URL = postgres://<username>:<pass>@localhost:5432/<db_name>

Now make a index.ts file inside db folder, and copy paste the lines in documentation.

Inside **scheme.ts**, lets make tables

```
import { pgTable, text, uuid, varchar } from 'drizzle-orm/pg-core'

export const usersTable = pgTable('users', {
    id : uuid('id').primaryKey().defaultRandom(),
    firstName : varchar('first-name', {length: 45}).
                                        notNull() ,
    lastName : varchar('last-name', {length: 45}),
    email : varchar('email', {length: 322}).notNull()
                                            .unique()
    password : varchar('password', {length: 66})
```

to make it required

to make sure user cannot give very long first name

320 char + 2 buffer

```
emailVerified : boolean ('email-verified').default(false).
                                          .notNull(),
salt : text('salt'),
createdAt : timestamp('created-at').defaultNow().
                                          notNull()
updatedAt : timestamp('updated-at').$onUpdate(() =>
                                          new Date()),

})
```

## Database Migration :

Add a script :
"studio" : "drizzle-kit studio"

1. Install drizzle-kit
2. Never just push : run drizzle-kit generate, so we can generate a SQL, review and edit it. When you are satisfied with the SQL, migrate it

Add script :
"db:generate" : "drizzle-kit generate",
"db:migrate" : "drizzle-kit migrate"

Now we move on to making routes.

To create salt,
1. Import
then
const salt = randomBytes(32).toString('hex')

#RainbowTables (common list of password), To make sure we append random string, so that even after hashing, the pass do not match the passwords in rainbow table.

const hash = .creatHmac('sha256', Salt).update
(password)
.digest('hex')

* Add same pass to DB, we need to store the salt to check what we appended. we can add salt prepend, postend, between characters to increase security.

The problem with stateless, if we want to remove some suspicious user, we can't store in db, but we can do a hybrid approach. do we keep access-token

If you read my notes follow x: jaaaani404

Assignment: go file by file and re-write code on your own.